Levelező gyakorlati ZH

Cím: Mini feladatkezelő alkalmazás

Időkeret: 90 perc

Technológia: Vue 3 + Composition API; a pluszfeladatokban API, Vue Router és Pinia.

Feladatleírás és értékelés

Készítsen egy „Teendőim” alkalmazást saját Vue-projektben. Ugyanezt bővítse a választott pluszfeladatokkal. Egyetlen projektet adjon be, amelyben az alapfeladat és a pluszfeladatok együtt működnek.

A működő alapfeladat elégséges (2). Minden teljesített pluszfeladat +1 jegy, legfeljebb jeles (5); a jeleshez bármely három pluszfeladat elegendő. Ha az alapfeladat nem működik, a ZH elégtelen (1), és egyetlen pluszfeladat sem értékelhető. A bővítések után is működjön a listázás, hozzáadás, validáció és állapotváltás.

Alapfeladat – helyi lista és űrlap

Az alapfeladat egy App.vue fájlban, ref alapú állapottal is megoldható. Az adatok kezdetben helyiek; az API bevezetésekor a szerverre kerülnek. Mezők: id, title és status (todo vagy done).

1. Jelenítsen meg két helyi kezdőfeladatot címükkel és állapotukkal, v-for és egyedi :key használatával. Az API bevezetésekor a lista adatait a szerverről töltse be.

2. Egy v-model mező és „Hozzáadás” gomb segítségével lehessen új feladatot felvenni. Az űrlap elküldése ne töltse újra az oldalt.

3. A cím elejéről és végéről vágja le a szóközöket. Üres címnél jelenjen meg hiba, és ne bővüljön a lista. Siker után ürüljön a mező és a hiba; az új elem egyedi id-t és todo állapotot kapjon.

4. Minden elem mellett legyen gomb, amely a todo és done állapotot váltja. A lista frissüljön azonnal.

Ellenőrzés: lista, szóközös cím elutasítása, hozzáadás és kétszeri állapotváltás. Ezek a pluszfeladatokkal együtt is működjenek. Olvasható megjelenés szükséges; különleges CSS nem követelmény.

1. pluszfeladat – szűrés és komponens (+1 jegy)

1. Készítsen összes / aktív / kész szűrőt v-model és computed segítségével. A szűrés a megjelenített listát változtassa, az eredeti tömböt őrizze meg.

2. Egy listasort emeljen ki TaskItem.vue komponensbe. A feladatot task propban kapja. A gomb toggle eseményt küldjön az id-val; a feladat állapotát a szülő módosítsa.

Ellenőrzés: mindhárom szűrő működik; üres találatnál üzenet látható; a gyermek eseményére a szülő listája és a szűrt nézet is frissül.

2. pluszfeladat – API-kapcsolat (+1 jegy)

Bővítse az alkalmazást szerveres adatkezeléssel: a helyi adatok helyett az API legyen az adatok forrása. Az API-hívásokat és a szükséges fájlokat készítse el fetch vagy Axios használatával. Az alapfeladat valamennyi funkcióját tartsa meg.

1. Az onMounted életciklus-eseménynél GET kéréssel töltse be a feladatokat, és jelenítse meg a kapott listát.

2. A betöltés alatt jelenjen meg „Betöltés…” üzenet. A sikeres, üres válasz és a sikertelen kérés külön üzenetet kapjon.

3. Az űrlap érvényes címét POST kéréssel mentse. Csak sikeres válasz után bővítse a listát a szerver által visszaadott elemmel, és ürítse a mezőt. Az id-t a szerver adja.

4. Mentés alatt a gomb legyen tiltott. Hibánál a bevitel maradjon meg, és jelenjen meg hibaüzenet. A loading és saving állapot minden esetben álljon vissza. fetch esetén ellenőrizze a response.ok értékét is.

5. Az API-hívásokat szervezze saját useApi() composable-ba, és azt használja a komponensből. A composable tartalmazzon lista-lekérdezést, létrehozást és módosítást végző függvényeket.

6. Az alapfeladat állapotváltó gombja PUT kéréssel mentse a módosított állapotot. Küldje el a feladat aktuális címét és új állapotát. Csak sikeres válasz után frissítse a listát; hibánál maradjon a korábbi állapot, és jelenjen meg hibaüzenet.

Rövid végpontleírás – lista, létrehozás, állapotváltás

Alapcím: https://zh.alitak.energo.cloud/api/<NEPTUNKOD>. A NEPTUNKOD helyére a saját Neptun-kódját írja. Az alábbi útvonalak ehhez az alapcímhez kapcsolódnak. A kérések és válaszok JSON formátumúak; a kérés törzsénél Content-Type: application/json fejléc szükséges.

Feladat: id (szerver által kiosztott pozitív egész), title (nem üres, trimelt szöveg), status (todo vagy done).

```
Példa válaszobjektum:
{ "id": 1, "title": "Beadandó elkészítése", "status": "todo" }
```

GET /tasks → 200: feladatobjektumok tömbje. Üres lista: []. A válasz közvetlen tömb, nincs data burkoló.

POST /tasks → kérés: { "title": "Új feladat" }; válasz: 201, a létrehozott teljes feladatobjektum, todo állapottal. id-t ne küldjön. PUT /tasks/:id → kérés: { "title": "Aktuális cím", "status": "done" }; válasz: 200, a mentett teljes feladatobjektum. Mindkét mező kötelező; az :id a feladat azonosítója.

Hibák: 404 hiányzó feladat; 422 érvénytelen cím vagy status; 500 szerverhiba. Formátum: { "message": "Hiba leírása" }. Hálózati hibánál is legyen visszajelzés. Az adatokat a szerver Neptun-kódonként elkülönítve kezeli.

3. pluszfeladat – Router, Pinia és teljes CRUD (+1 jegy)

A meglévő alkalmazás szerveres adatkezelését bővítse nézetekkel, routerrel és Pinia store-ral. A lista, az űrlap és az állapotváltás továbbra is legyen használható ugyanebben a projektben.

1. Készítsen /tasks listaoldalt és /tasks/:id részletoldalt. A listából RouterLink segítségével lehessen a részletekre lépni; a részletoldalon legyen visszalépési lehetőség. A nézeteket RouterView jelenítse meg.

2. A feladatok közös állapotát helyezze Pinia store-ba. Legyen tasks állapot, feladatszámot adó getter, valamint betöltő, létrehozó, módosító és törlő action. Az API-kommunikációhoz használja a saját composable-ját.

3. A részletoldalon GET kéréssel töltse be az adott feladatot. Lehessen a címét és állapotát módosítani, majd PUT kéréssel menteni. Siker után a store a szerver válaszát tárolja.

4. Készítsen törlés gombot confirm() megerősítéssel. DELETE csak jóváhagyás után induljon. Siker után frissüljön a lista, és az alkalmazás navigáljon a /tasks oldalra.

5. Betöltés, módosítás és törlés közben legyen megfelelő visszajelzés és hibakezelés. Hiányzó feladatnál jelenjen meg „A feladat nem található.” üzenet; ismeretlen alkalmazásútvonalhoz készítsen hibaoldalt.

Rövid végpontleírás – részlet, módosítás, törlés

Az alapcím és a feladatmodell megegyezik a 2. pluszfeladatban megadottal. Az :id helyére a feladat számértékű azonosítója kerül.

GET /tasks/:id → 200, egy teljes feladatobjektum; hiányzó azonosítónál 404.

PUT /tasks/:id → kérés: { "title": "Módosított cím", "status": "done" }; válasz: 200, a mentett teljes feladatobjektum. Mindkét szerkeszthető mező kötelező; az id változatlan marad.

DELETE /tasks/:id → 204, üres válasz. A 204-es válaszhoz ne próbáljon JSON törzset feldolgozni.

Hibák: 404 hiányzó feladat; 422 érvénytelen cím vagy status; 500 szerverhiba. A hibaüzenet formátuma azonos a 2. pluszfeladatban leírttal.

Ellenőrzés: listából megnyitott és közvetlenül elért részletoldal; mentett cím és állapot; visszautasított, majd jóváhagyott törlés; frissülő store és lista; hiányzó azonosító és hibás URL.

4. pluszfeladat – bejelentkezés és védett profil (+1 jegy)

Az alkalmazásban készítsen /login és /profile nézetet, valamint Pinia auth store-t. A profil legyen elérhető az alkalmazás navigációjából; a feladatlista belépés nélkül is működjön. A tesztfelhasználó belépési adatai a ZH-n kerülnek megadásra.

1. A bejelentkező űrlap username és password mezőt tartalmazzon. POST kéréssel küldje az adatokat a login végpontra. Siker után a tokent és a felhasználót tárolja az auth store-ban, majd navigáljon a profilhoz.

2. Router guard segítségével csak bejelentkezett állapotban lehessen a /profile nézetet megnyitni. Egyébként a /login oldal jelenjen meg.

3. A profil adatait a /auth/me végpontról töltse be. A védett kérésekhez küldje el a tokent Authorization: Bearer <token> fejlécben. Jelenítse meg a felhasználó nevét.

4. Készítsen kijelentkezés gombot, amely meghívja a logout végpontot. Siker után törölje az auth store tokenjét és felhasználóját, majd lépjen vissza a bejelentkezéshez.

5. Hibás belépésnél jelenjen meg hiba, és ne nyíljon meg a profil. A /auth/me 401 válaszánál törölje a belépési állapotot, és navigáljon a /login oldalra. Kérés közben legyen betöltési jelzés és gombtiltás.

A token memóriában tárolása elegendő; oldalfrissítés után új bejelentkezés kérhető. Az auth store tartalmazzon felhasználói állapotot, isAuthenticated gettert, valamint login, loadProfile és logout actionöket.

Rövid végpontleírás – autentikáció

Az alapcím azonos a 2. pluszfeladatban megadottal. A profil végpontja szerveroldali tokenellenőrzést végez. A /tasks végpontok a többi pluszfeladatban bejelentkezés nélkül használhatók.

POST /auth/login → kérés: { "username": "…", "password": "…" }; válasz: 200, token és felhasználó:

```
{ "token": "…",
  "user": { "id": 1, "name": "Teszt Elek", "username": "teszt" } }
```

GET /auth/me → Bearer token szükséges; válasz: 200, a fenti user objektum közvetlenül, user burkoló nélkül.

POST /auth/logout → Bearer token szükséges; nincs kérés-törzs; válasz: 204, üres törzs. A szerver érvényteleníti a tokent.

Hibák: 401 hibás belépési adatok, hiányzó vagy érvénytelen token; 422 hiányzó username vagy password; 500 szerverhiba. Formátum: { "message": "Hiba leírása" }.

Ellenőrzés: hibás és sikeres belépés; profil elérése belépés nélkül; tokenes profilbetöltés; kijelentkezés és 401 kezelése. Belépés előtt és kijelentkezés után is működjön az alapfeladat ugyanabban az alkalmazásban.